

Backend Developer Guide: Biometric Face-recognition

Project Name: FARO (Face-base Access and Room Observer)

Framework: Fastapi + Docker + Postgres + Pgadmin + Pgvector

Introduction & Overview

1.1 โปรเจกต์คืออะไร? (What is this?)

Face-base Access and Room Observer คือ Web Application ประกอบไปด้วย Enrollment Verification Identification สำหรับระบบยืนยันตัวตนด้วยใบหน้า (Biometric Face Authentication) และระบบตรวจจับ/ระบุตัวบุคคลแบบ Real-time ภายในห้องเรียนหรือพื้นที่ควบคุม

ระบบทำงานโดย:

- เปิดกล้องผ่าน Browser (WebRTC)
- ส่งภาพแบบ real-time ผ่าน WebSocket
- Backend ประมวลผลด้วย AI (Face Detection + Face Recognition)
- ส่งผลลัพธ์กลับมาแสดงบนหน้าเว็บทันที

1.2 วัตถุประสงค์ของระบบ (System Objectives)

- ยืนยันตัวตนผู้ใช้ด้วยใบหน้า
- ตรวจจับและระบุตัวบุคคลแบบ Real-time
- แสดงผลจำนวนคนเข้า-ออกห้อง
- รองรับการใช้งานผ่าน Browser ทั้ง Desktop และ Mobile

1.3 Backend มีหน้าที่อะไร?

1.3.1 API Server

- จัดการ Register / Login
- ตรวจสอบ JWT Token
- จัดการ Profile ผู้ใช้
- จัดการสิทธิ์ (Authorization)

1.3.2 Real-time Face Recognition Engine

- รับภาพจาก WebSocket

- Detect ใบหน้า
- Extract embedding
- ค้นหา vector ใกล้เคียงในฐานข้อมูล
- ส่งผลลัพธ์กลับ Frontend

1.3.3 Database Manager

- จัดเก็บข้อมูลผู้ใช้
- จัดเก็บ face embedding
- บันทึก log การเข้า-ออก

1.3.4 Vector Search System

- ใช้ pgvector คำนวณ cosine similarity
- ค้นหา embedding ที่ใกล้ที่สุด

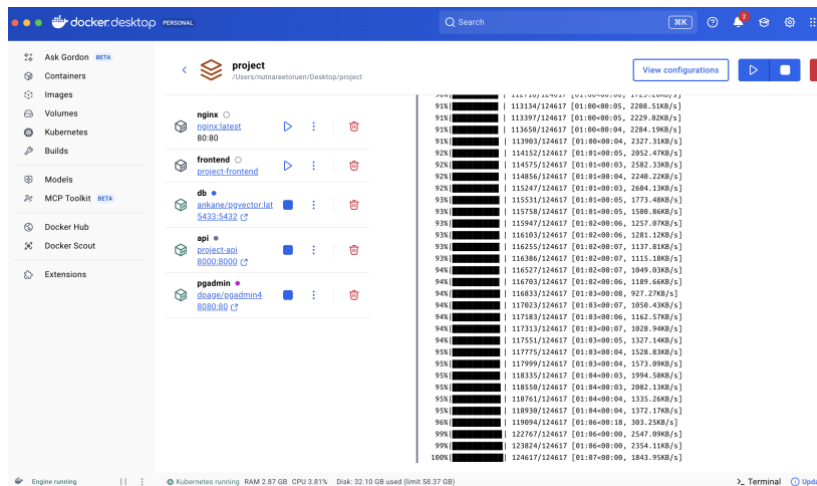
2. Backend Architecture

2.1 Docker ใช้สำหรับ:

- Containerize ระบบทั้งหมด
- แยก service ออกจากกัน
- ควบคุม environment ให้เหมือนกันทุกเครื่อง
- Deploy ได้ง่าย

ประกอบด้วย service หลัก:

- api (FastAPI)
- db (PostgreSQL)
- pgadmin



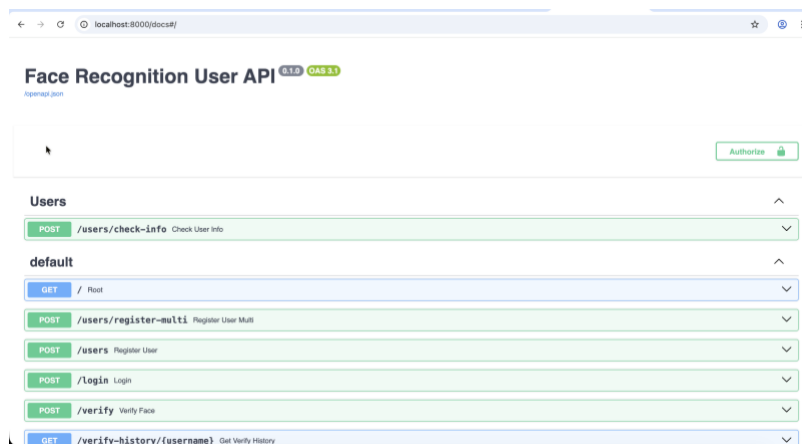
รูปภาพประกอบข้อ 2.1

2.2 FastAPI ใช้สำหรับ:

- สร้าง REST API
- สร้าง WebSocket endpoint
- จัดการ Authentication (JWT)
- เชื่อมต่อ Database ด้วย SQLAlchemy
- ประมวลผล AI model

ข้อดี:

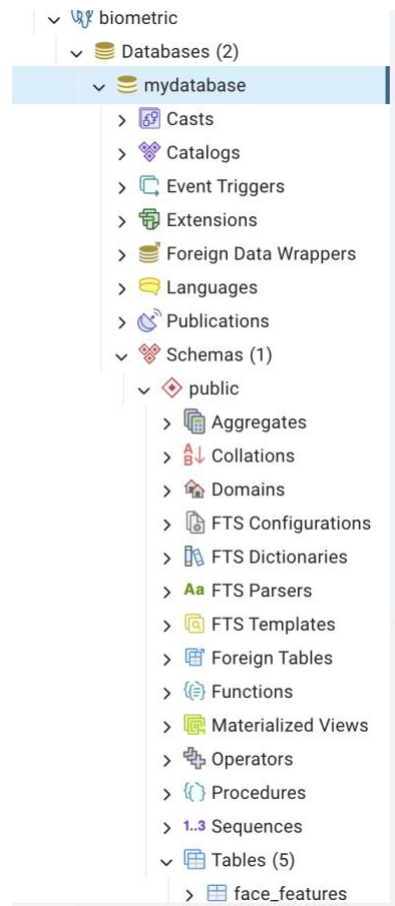
- รองรับ async
- Performance สูง
- เหมาะกับ real-time application



รูปภาพประกอบข้อ 2.2

2.3 PostgreSQL ใช้สำหรับ:

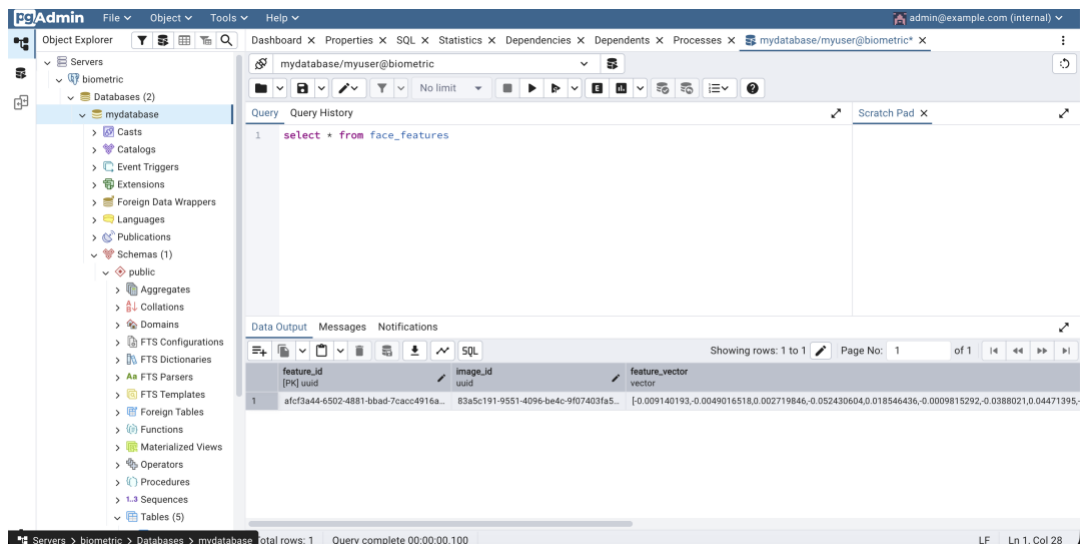
- เก็บข้อมูลผู้ใช้ (User table)
- เก็บ Face Embedding
- เก็บ Log การเข้า-ออก
- เก็บข้อมูล session



รูปภาพประกอบข้อ 2.3

2.4 pgAdmin ใช้สำหรับ:

- บริหารจัดการฐานข้อมูลผ่าน GUI
- ตรวจสอบ table
- ดู vector embedding
- Debug query

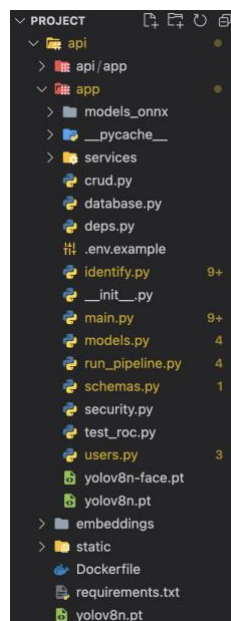


รูปภาพประกอบข้อ 2.4

2.5 Pgvector ใช้สำหรับ:

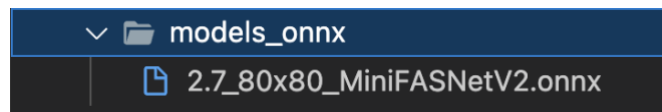
- เก็บ vector embedding (เช่น 512 dimension)
- ทำ similarity search
- รองรับ cosine distance / L2 distance

3. โครงสร้างโปรเจก (Project Structure)



(รูปภาพประกอบข้อ3)

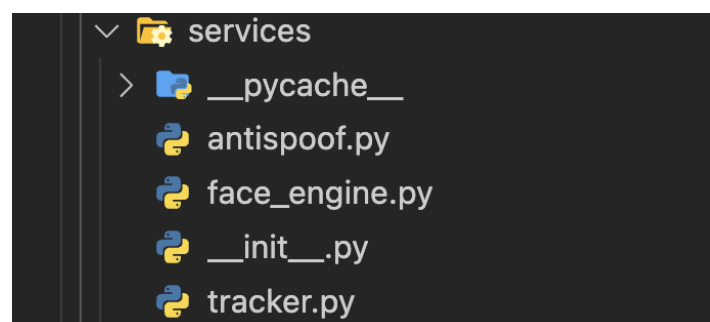
3.1 Folder models_onnx คือ model สำหรับการป้องกันใบหน้าปลอมหรือ Spoofing



(รูปภาพประกอบข้อ3.1)

3.2 Folder Sevice จะมีไฟล์ต่างๆดังนี้

1. antispoof.py ใช้สำหรับ Loadmodel spoofing และconfig จากข้อ 3.1
2. face_engine.py ใช้สำหรับ Loadmodel Buffalo_s จาก insighface
3. tracker ใช้สำหรับติดตามใบหน้า



(รูปภาพประกอบข้อ3.2)

3.3 ไฟล์หลัก

3.3.1 Core & Configuration (พื้นฐานระบบ)

database.py: ใช้เชื่อมต่อกับฐานข้อมูล PostgreSQL โดยดึงค่า Config (User, Password, Host) มาจากไฟล์ .env และสร้าง SessionLocal เพื่อให้ไฟล์อื่นนำไปใช้ดึงข้อมูล

models.py: กำหนดโครงสร้างตาราง (Table) ในฐานข้อมูล เช่น ตารางผู้ใช้ (User), ตารางเก็บรูปภาพ (FacelImage), และตารางเก็บ Vector ใบหน้า (FaceFeature) ที่ใช้ **pgvector**

schemas.py: ใช้กำหนดรูปแบบข้อมูล (Pydantic) สำหรับการรับ-ส่งผ่าน API และใช้ตรวจสอบความถูกต้อง (Validation) เช่น ตรวจสอบว่า Username ต้องเป็นภาษาอังกฤษ หรือรหัสผ่านต้องมี 8 ตัวขึ้นไป

3.3.2 API Endpoints & Business Logic (ส่วนควบคุมหลัก)

main.py: ไฟล์หลักที่รันระบบ FastAPI ทั้งหมด รวบรวม Route ต่างๆ, จัดการการสมัครสมาชิกแบบ 3 รูปภาพ (register-multi), ระบบ Login, และการตรวจสอบใบหน้า (verify)

users.py: จัดการเส้นทาง (Route) เกี่ยวกับข้อมูลผู้ใช้โดยเฉพาะ เช่น API สำหรับตรวจสอบว่า Username หรือ Email ซ้ำหรือไม่ก่อนการสมัคร

crud.py: ย่อมาจาก Create, Read, Update, Delete เป็นไฟล์รวมฟังก์ชันที่ใช้ติดต่อกับ Database โดยตรง (เช่น การสร้าง User ใหม่ หรือดึงข้อมูล Feature จากฐานข้อมูล)

3.3.3 Security & Authentication (ความปลอดภัย)

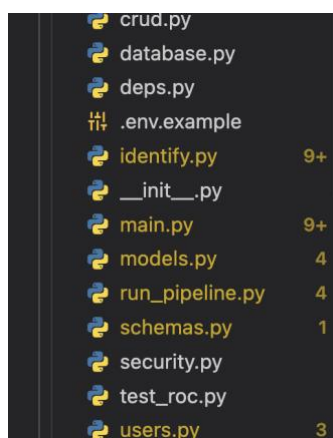
security.py: ใช้สำหรับเข้ารหัสผ่าน (Hashing) ก่อนเก็บลง Database และตรวจสอบรหัสผ่านเมื่อมีการ Login

deps.py: ย่อมาจาก Dependencies เป็นไฟล์รวมฟังก์ชันเสริมที่ต้องเรียกใช้บ่อยๆ เช่น การเปิด-ปิด Database Connection และการตรวจสอบสิทธิ์ของผู้ใช้งาน (JWT Token / Admin Check)

3.3.4 AI & Real-time Processing (ส่วนประมวลผล AI)

identify.py: จัดการระบบ Real-time Face Recognition ผ่าน WebSocket โดยใช้ Multiprocessing เพื่อให้ประมวลผลภาพจากกล้องหลายตัวได้พร้อมกัน และมีระบบ Line Crossing เพื่อตรวจคนเข้า-ออกห้อง

run_pipeline.py: รวบรวมฟังก์ชันการทำงานของ AI เช่น การตรวจจับใบหน้า (Detection), การทำความเข้าใจ (Alignment), และการสกัดคุณลักษณะ (Feature Extraction) เพื่อนำไปใช้งานใน API ส่วนต่างๆ



(ภาพประกอบข้อ3.3)

คำแนะนำเพิ่มเติม:

ถ้าต้องการแก้ Database: ให้ไปดูที่ models.py และ database.py

ถ้าต้องการแก้ Logic การตรวจจ็บบหน้า: ให้ดูที่ identify.py และ run_pipeline.py

ถ้าต้องการเพิ่ม API ใหม่: ให้สร้าง Route ใน users.py หรือ main.py แล้วเขียนฟังก์ชันจัดการข้อมูลใน crud.py

4. Security Guideline

- ใช้ bcrypt หรือ argon2 hash password
- JWT ต้องมี:
 - exp
 - iat
 - role
- Access token / Refresh token แยกกัน
- Rate limit API login
- WebSocket ต้อง verify JWT ก่อน connect
- จำกัด file size ของภาพ
- CORS ต้องกำหนด origin ชัดเจน
- ปิด pgAdmin ใน production

5. การติดตั้งและเริ่มใช้งาน (Backend Setup Guide)

ส่วนนี้สำคัญมากเพื่อให้้องสามารถจำลอง Environment ให้เหมือนกับพีได้:

- Prerequisites:
 - ติดตั้ง Docker Desktop (สำหรับจัดการ Container ของ Database และ API)
 - Python 3.10+ (กรณีต้องการรันแบบ Local สำหรับ Debug เฉพาะไฟล์)
- Step-by-Step Setup:
 1. **Environment Setup:** Copy ไฟล์ .env.example แล้วเปลี่ยนชื่อเป็น .env เพื่อตั้งค่า DB_USER, DB_PASSWORD ให้ตรงกับที่กำหนดใน database.py

2. **Docker Launch:** เปิด Terminal ในโฟลเดอร์โปรเจกต์แล้วรันคำสั่ง `docker-compose up --build` เพื่อสร้างและรัน Service ทั้งหมด (API, DB, pgAdmin)
3. **Database Migration:** ระบบจะสร้างตารางและ Extension `pgvector` ให้โดยอัตโนมัติเมื่อรันครั้งแรกผ่าน `on_startup` ใน `main.py`
4. **Verification:** เข้าไปที่ <http://localhost:8000/docs> เพื่อเช็คค่า API พร้อมใช้งานหรือไม่

6. โฟลว์การทำงานของระบบ (System Workflows)

เพื่อให้น้องไม่มองว่าไฟล์ไหนคุยกับไฟล์ไหน พี่สรุปขั้นตอนหลักๆ ไว้ให้ครับ:

6.1 การลงทะเบียนแบบหลายรูป (Register Multi-Image Workflow)

1. **Frontend** ส่งรูปภาพ 3 รูป (หน้าตรง, หันซ้าย, หันขวา) มาที่ `/users/register-multi`
2. **Backend (main.py)** เรียกใช้ฟังก์ชันจาก `run_pipeline.py` เพื่อ Detect ใบหน้าและสกัด Vector
3. ระบบตรวจสอบความซ้ำซ้อนของใบหน้า (Face Duplication Check) ในฐานข้อมูลโดยใช้ `pgvector`
4. ถ้าผ่าน ข้อมูลจะถูกบันทึกลง Table `users`, `face_images`, และ `face_features` ผ่านทาง `crud.py`

6.2 การตรวจจับบุคคลแบบ Real-time (Real-time Identification)

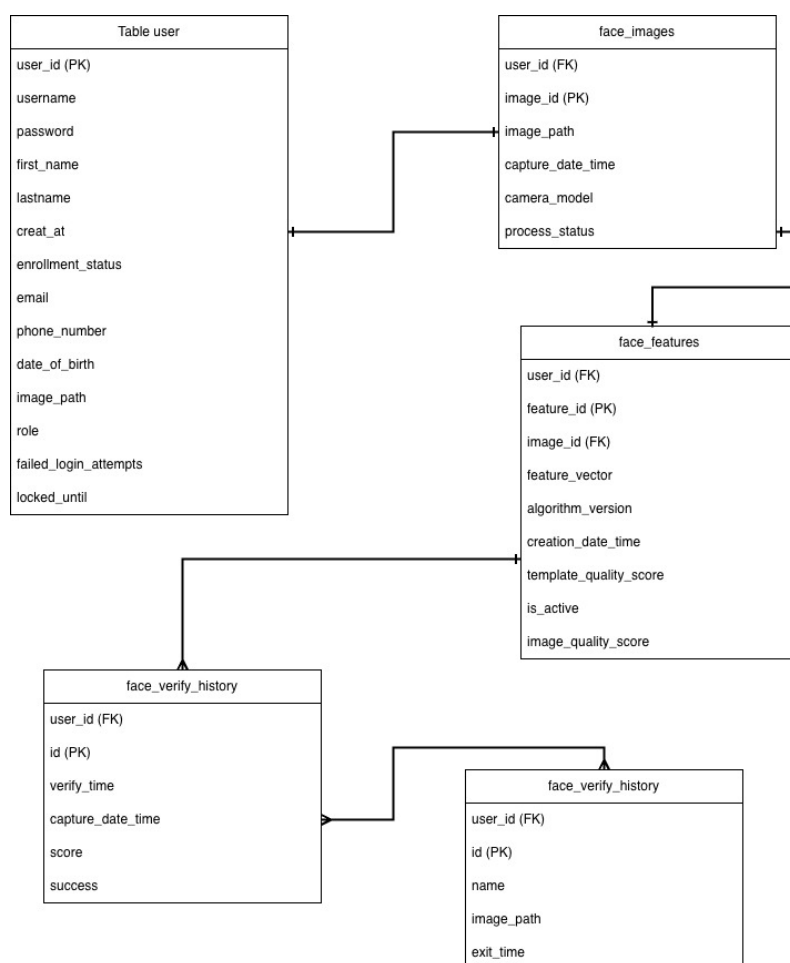
1. **Frontend** เชื่อมต่อ WebSocket มาที่ `/ws/identify` พร้อมส่ง JWT Token
2. `identify.py` รับเฟรมภาพแล้วส่งเข้า **Worker Process** เพื่อไม่ให้ระบบหลักหน่วง
3. **AI Engine** ทำการ Track บุคคล (YOLO) และ Recognize ใบหน้า (InsightFace)
4. **Line Crossing Logic:** ตรวจสอบว่าพิกัดใบหน้าข้ามเส้นสมมติที่กำหนดไว้หรือไม่ เพื่อบันทึกลง `visitor_logs`
5. ส่ง JSON ผลลัพธ์ (ชื่อคน, คะแนนความมั่นใจ, สถานะการเข้า-ออก) กลับไปให้ Frontend ทันทที

7. แผนพัฒนาในอนาคต (Future Roadmap)

น้องสามารถสานต่อโปรเจกในหัวข้อเหล่านี้ได้เลย:

- **Performance Optimization:** ปรับปรุงการใช้ Multiprocessing ใน identify.py ให้รองรับกล้องจำนวนมากขึ้น
- **Advanced Anti-Spoofing:** พัฒนาโมเดลในโฟลเดอร์ models_onnx ให้แม่นยำขึ้นเพื่อป้องกันการใช้รูปภาพหลอกกล้อง
- **Logging System:** ขยายระบบการเก็บ Log ให้รองรับการ Export ข้อมูลออกมาเป็นไฟล์ Excel หรือ Report รายเดือน

8. database design



9. เหตุผลหลักที่โปรเจก FARO เลือกใช้โมเดล YOLOv8n และ InsightFace (buffalo_s)

เป็นเพราะความสมดุลระหว่าง "ความเร็ว" และ "ความแม่นยำ" ซึ่งจำเป็นมากสำหรับระบบ Real-time identification โดยมีรายละเอียดดังนี้:

9.1 YOLOv8n (Nano)

ใช้สำหรับการตรวจจับบุคคล (Body Detection) และติดตาม (Tracking):

ความเร็วสูง (High Performance): YOLOv8n เป็นโมเดลเวอร์ชันที่เล็กที่สุด (Nano) ในตระกูล v8 ทำให้ประมวลผลได้เร็วมากแม้จะรันบน CPU.

ใช้ทรัพยากรน้อย: เหมาะสำหรับการทำ Multiprocessing หรือรันใน Docker Container โดยไม่กินแรมจนเครื่องค้าง.

การติดตามที่แม่นยำ: มีระบบ Tracking ในตัว (เช่น BoT-SORT หรือ ByteTrack) ซึ่งจำเป็นต่อระบบ Line Crossing เพื่อดูว่าคนไหนกำลังเดินเข้าหรือเดินออก.

9.2 InsightFace (buffalo_s)

ใช้สำหรับการระบุตัวตนใบหน้า (Face Recognition):

Buffalo_s (Small): เป็นโมเดลขนาดเล็กที่ออกแบบมาเพื่อให้รันได้บนไลบรารี CPU แต่ยังคงความแม่นยำที่สูงมากเมื่อเทียบกับโมเดลขนาดเดียวกัน.

Sub-models ในตัว: ภายในชุด buffalo_s ประกอบด้วยโมเดลย่อยที่ทำงานร่วมกัน ทั้งการตรวจจับตำแหน่งใบหน้า (Detection), การหาจุดสำคัญบนหน้า (Keypoints), และการสร้าง Vector (Embedding).

มาตรฐานระดับสากล: InsightFace เป็น Library ที่ได้รับความนิยมสูงสุดในวงการวิจัยใบหน้า ทำให้หาเอกสารหรือวิธีแก้ปัญหาได้ง่ายสำหรับน้องที่ตามมาทำต่อ.

9.3 ทำไมต้องใช้คู่กัน?

การแยกงานกันทำช่วยให้ระบบมีประสิทธิภาพสูงสุด (Optimization):

YOLOv8n ทำหน้าที่ "สแกนหาคนทั้งภาพ" เพื่อระบุขอบเขตบุคคล.

InsightFace จะทำงานเฉพาะ "ในพื้นที่เล็กๆ ที่เจอใบหน้า" เพื่อสกัด Vector 512 มิติ.

Pgvector นำ Vector ที่ได้ไปค้นหาในฐานข้อมูลด้วย Cosine Similarity เพื่อระบุว่าเป็นใคร.

สรุปสั้นๆ: การเลือกโมเดลตระกูล "n" (Nano) และ "s" (Small) ช่วยให้ระบบ FARO สามารถทำงานแบบ Real-time ได้จริงบนเครื่องคอมพิวเตอร์ทั่วไป โดยไม่ต้องพึ่งพาคาร์ดจอ

ช่องทางการติดต่อ หากติดขัดหรือมีเรื่องสงสัย

Email : f.panuthep@gmail.com | Ig : fpnt.clv | Fb : panuthep chulavech